

The Impact of Player Behavior Features on Game Performance Outcomes

Roni Roitbord

Technion – Israel Institute of Technology
roniro@campus.technion.ac.il
CS Department

Yagel Maimon

Technion – Israel Institute of Technology
myagel@campus.technion.ac.il
CS Department

Salih Hassan

Technion – Israel Institute of Technology
salihhassan@campus.technion.ac.il
CS Department

Winter 24-25

1 Introduction

Git repo: <https://github.com/Av0cat0/Clash-Royale-Causal-analysis>

The gaming industry has grown into a significant sector, with multiplayer online games being a dominant force. Understanding the factors that influence player success is critical for designing balanced gameplay and fostering player engagement. Techniques such as feature correlation, machine learning models, and dimensionality reduction are commonly used to understand player behaviors. For example, metrics like actions per minute (APM) and teamwork frequency have been correlated with higher win rates in eSports titles [6, 7].

1.1 Research Question

This study aims to present the gap between raw data and actionable insights by identifying key behavioral features. Our goal is to identify and analyze the impact of player behavior features on game performance outcomes. Leveraging gaming datasets from Kaggle, this study aims to enhance decision-making processes in game design and player analytics by detecting key behavioral factors that influence player success. Based on our goal and motivation, we can define our research question as

Which player behavior features most impact game performance, and how can we model them for strategic insights?

The first issue we encountered with was a huge hassle to deal with: the game is played by approximately

35M people each month. The game is maintained and refined to the level that every patch contains a character balancing factors, which can bias our research toward old data in the long run. Our approach to solve this issue is to make the code as much as **abstract as possible**, mainly by retrieving the datasets and then analyzing it based on global game mechanics and metrics.

1.2 Game Mechanics and Rules

Clash Royale is a real-time strategy game that combines elements of collectible card games and tower defense. Players compete in 1vs1 or 2vs2 battle, using decks of eight cards that summon troops, spells, or buildings onto the battlefield. The **objective** is to destroy the opponent's King Tower while defending one's own.

Each battle takes place on a two-lane battlefield, where players deploy units that automatically move toward the enemy towers. A match lasts three minutes, with an additional overtime period if neither player has destroyed the King Tower. During the final 60 seconds, elixir generation doubles, allowing for faster card deployment. Key game mechanics include:

- **Elixir Management:** Players have a regenerating resource called **Elixir**, which is used to deploy cards. Efficient Elixir usage is critical for maintaining pressure and countering opponent strategies.
- **Card Rarity and Levels:** Cards come in four rarities—**Common**, **Rare**, **Epic**, and **Leg-**

endary. Higher-level cards provide stat advantages, influencing battle outcomes.

- **Troop Types and Roles:** Cards categorized into **troops, spells, and buildings**, each serving offensive/defensive purposes. Some cards synergize well, creating effective push strategies.
- **Win Conditions:** Players often structure their decks around a "win condition" (e.g., Hog Rider, Balloon, Graveyard) that serves as their primary method of dealing tower damage.
- **Trophy-Based Ranking System:** Players gain or lose **trophies** based on match outcomes, progressing through different arenas and leagues.

Success in Clash Royale requires a combination of **strategic deck-building, real-time decision-making, and resource management.** Understanding these mechanics is crucial for analyzing how behavior and deck choices impact game performance.

2 EDA

2.1 Data Acquisition

To conduct our EDA, we first acquire datasets from Kaggle [1, 4] using an automated Python script, ensuring reproducibility and ease of access. The dataset contains structured information on Clash Royale battles, including player attributes, deck compositions, and game outcomes. The data is downloaded using the `download_kaggle_datasets()` function from our preprocessing script, which retrieves multiple tables but focuses primarily on the battle logs. These logs provide a comprehensive view of player interactions, including the winner and loser's decks, elixir costs, and match statistics. The dataset structure is further explored in our Jupyter Notebook, where key variables such as `winner_deck_final_score` and `deck_weighted_strength` are visualized and analyzed to understand player performance dynamics. The collection spanned an entire season, from early December 2020 until the conclusion of Season 18.

2.1.1 Data Structure

The analysis is based on three interconnected datasets: the **battle logs**, the **card master list**, and the **achieved-win conditions dataset**.

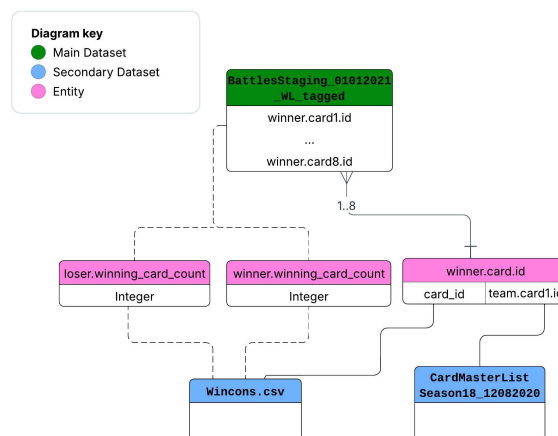


Figure 1: Data Structure

BattlesStaging_01012021_WL_tagged.csv: A comprehensive log of individual battles, including player decks, battle outcomes, and metadata. Each deck in this dataset is represented as a one-hot vector. This vector encodes the presence or absence of all available cards, each entry in the vector corresponds to a unique **Card ID** from the identification table, where a value of 1 indicates the presence of a card corresponding to an index and 0 means its absence. Since each deck consists of 8 cards from the entire pool, most entries in the binary vector are zeroes, making it a sparse representation. This structure allows for efficient comparisons between decks and enables computing similarity metrics between decks and assessing the strength of a deck and its synergy.

CardMasterListSeason18_12082020.csv: A reference table mapping card IDs to their names.

Wincons.csv: A subset of the reference dataset, containing cards that are game changers and could cause a player to win. This table is used to classify decks based on their reliance on these specific cards.

Dataset Integration: The datasets are connected through the **card IDs** present in both the battle logs and the card master list. Each player's deck is mapped to its corresponding card attributes using these IDs. The integration process allows us to engineer meaningful features, such as deck synergy and win condition frequency, which are later used in the exploratory data analysis and modeling phases.

2.2 Exploratory Data Analysis

2.2.1 Simpson's Paradox

- 2 A key challenge we experienced in interpreting player performance data is the potential presence of Simpson's Paradox, where trends observed in aggregated

data would disappear or even reverse when examined within subgroups. For example, a naive analysis might suggest that decks with a high **elixir cost** have lower win rates. However, when stratified by **arena level** or **player experience**, we could find that **high-elixir decks** perform exceptionally well among less experienced players. This paradox highlights the importance of considering confounding variables such as playstyle, card synergy, and other metrics when analyzing deck effectiveness. To mitigate this, we employ DAG to uncover the true relationships between features, ensuring that our conclusions reflect meaningful patterns rather than misleading correlations.

2.2.2 DAG

Introduction: We constructed the DAG using three methods: **Apriori**, **FP-Growth**, and **correlations**. To run the apriori and FP-growth algorithm first, we extracted binary columns and transformed non-binary ones by setting thresholds based on Apriori frequencies on each column. We then applied Apriori and FP-Growth with varying support and confidence levels, refining the DAG. Finally, we used correlation matrix for further insights. In the transaction-based algorithms, we found connections between `winner.win_rate`, `rarity_diversity`, `winner.win_rate` and `synergy_score`, Which can be used when building our scores.

Apriori vs. FP-Growth Algorithm: Both methods yielded the same results, but as we learned in class, FP-Growth was significantly faster because it using the FP-tree Data structure. Apriori took one hour for 100 rows, while FP-Growth processed 10,000 rows in 50 minutes, confirming its efficiency.

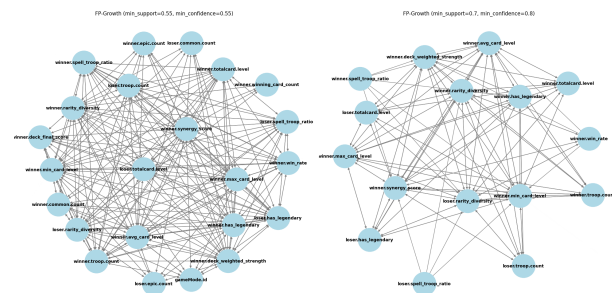


Figure 2: create DAG using FP-growth with different configuration

FP-Growth vs. Correlation Matrix: Correlation-based DAGs were more informative than those from FP-Growth, revealing deeper feature relationships. Using a 0.55 correlation

threshold, we created a structured DAG with meaningful insights.

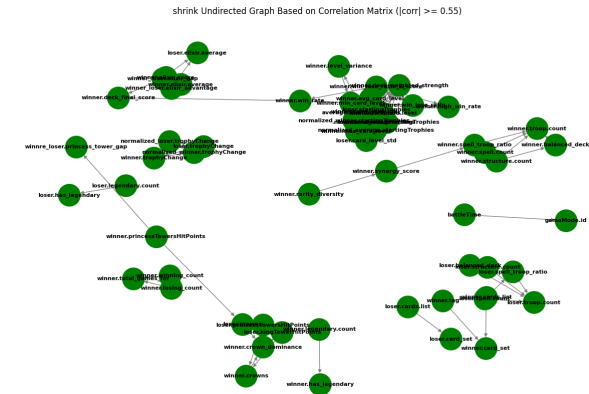


Figure 3: Full DAG

Methodology: We began by constructing a DAG iteratively throughout our research, refining it over time. At each stage, we incorporated feature engineering manipulations to enhance the representation of key relationships. This process allowed us to gradually uncover meaningful dependencies and improve the accuracy of our insights. we tried at first to make the correlation of **0.7** and above but we saw that all the connections were obvious, then we tried edges based on a correlation of **0.4** and above and saw that we had too many edges that we couldn't conclude enough from it. At least we tried to add edges only when the correlation is **0.55** and above and saw interesting dags that we could understand a lot from.

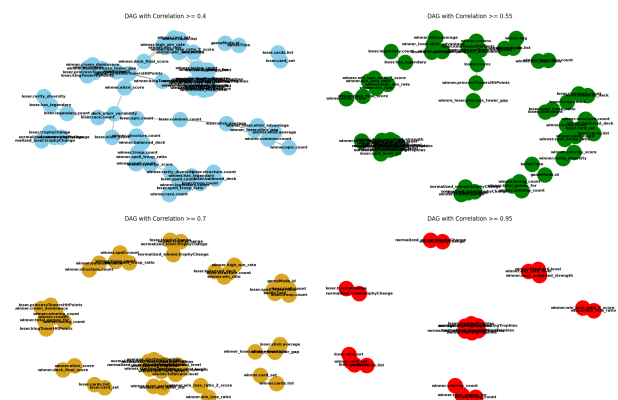


Figure 4: Different Correlations DAGs

Focus on the DAG with edges based on **0.55** correlation. Initially, we observed many correlated features, resulting in a complex DAG with seven connected components.

Our DAG has 13 connected components, each logical and interesting, and was created by our correlation matrix after the correlation parameter tuning. We will analyze a couple of our connected components and will design the direction of each edge by leveraging our domain knowledge.

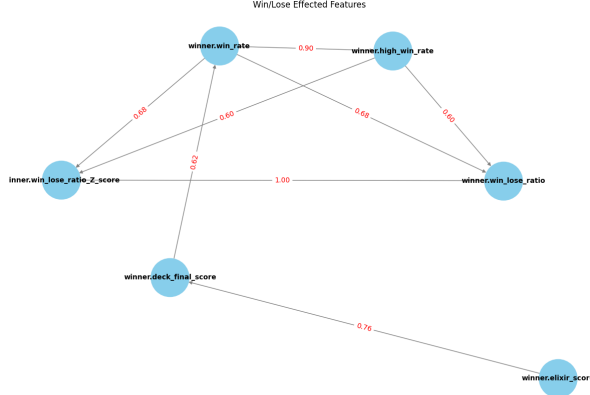


Figure 5: Win-Lose Ratio Dependencies

This graph highlights that the `win-lose ratio` is significantly influenced by the `win_rate` feature we created that is based on the `winner.card.set`. The second subgraph will be the winners and losers' crown-earned dependents:

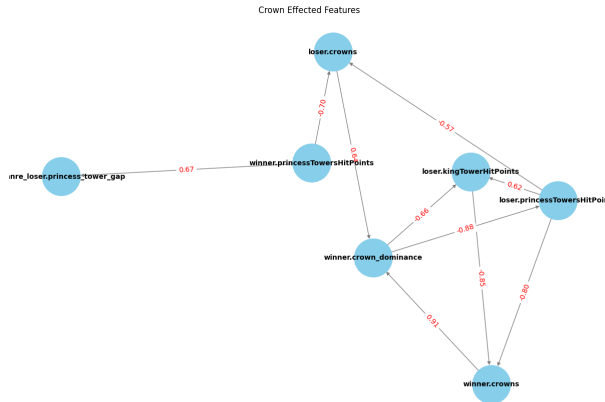


Figure 6: Crown Earned Dependencies

This graph reveals that the number of crowns earned is strongly dependent on the tower hitpoints of both the King and Princess towers. These findings offer new perspectives on gameplay strategies, particularly the importance of deck selection and player experience in determining match success. Future work will focus on further refining these dependencies to develop actionable insights for competitive play.

- Insight 1: The key factor influencing the win/loss ratio (long-term outcome) is selecting the right deck based on its final score, which impacts the win rate. The win rate serves as a "black box" for evaluating the deck's effectiveness, while the deck's final score acts as a "white box," guiding the card selection process to replicate optimal choices for winning.
- Insight 2: To earn the maximum number of crowns (immediate outcome), we need to deal as much damage as possible to the opponent's Princess and King Towers.

Conclusion: We started by identifying the optimal correlation threshold to determine which features have the most significant influence. We found that a threshold of **0.55** strikes the right balance—it avoids highlighting only obvious connections while preventing excessive links between too many features.

Next, we investigated each connected component and categorized outcomes into two types: immediate and long-term. Our analysis revealed a strong correlation between our `deck_final_score` formula and long-term success. Additionally, we observed that maximizing damage to the Princess and King Towers greatly increases the chances of achieving the best immediate outcome.

2.3 Data Preparation and Cleaning

2.3.1 Imputation

Incomplete data can affect our accuracy of analysis, particularly in our high-dimensional datasets. To address those missing values, we apply a structured imputation strategy tailored to different feature types:

- For `loser.kingTowerHitPoints`, we use a **fill strategy** to replace missing values with a pre-defined constant to ensure that missing entries do not interfere with similarity calculations.
- For numerical features such as `loser.princessTowersHitPoints` and `winner.princessTowersHitPoints`, we apply a similar **fill strategy**, with the latter explicitly set to [0] to maintain the expected data format.

2.3.2 Feature engineering

To gain meaningful insights into the relationship between player behavior and game performance, we engineered several features from the raw dataset. These features were designed to capture various aspects of

deck strength, player performance, and card effectiveness. The key feature categories are outlined below:

Deck Strength Features. To evaluate the strength of each player’s deck, we computed weighted strength metrics that incorporate both card levels and historical win rates: We’ll denote Deck Weighted Strength as **DWS**, where it is A weighted sum of individual card strengths, and each card’s contribution is computed as:

$$DWS = \sum_{i=1}^8 (\text{win_rate}_i \times \text{card_level}_i) \quad (1)$$

where win_rate_i represents the historical win rate of card i at its given level.

Card Level Statistics. Since card levels significantly influence game outcomes, we computed the following statistical metrics:

- **Average, Min and Max Card Level:** The mean/max/min level of all cards in a deck.
- **Card Level Variance:** Measures variation in card levels, where a lower variance suggests a more balanced and defined deck.

Elixir and Rarity-Based Features. Elixir cost and rarity of cards play a crucial role in deck efficiency. We introduced:

- **Elixir Variability:** The standard deviation of elixir costs across all cards in a deck.
- **Rarity Diversity:** Counts different rarities (Common/Rare/Epic/Legendary) within a deck.

Player Performance Metrics We derived metrics based on player skill and game progression:

- **Starting Trophies:** A value representing the player’s initial trophy count.
- **Trophy Change:** The difference in trophies before and after a match.
- **Win-Loss Ratio:** Computed as the total number of wins divided by total losses for each player.
- **Winner Spell Troop Ratio:** A ratio of the spell VS. the troops counts, used for the **balancing factor** of the deck (explained beneath)

- **Synergy Score:** Checks the balancing factor according to the domain knowledge of how a factored deck should be constructed. The deck will contain either one building card, two building cards or none (as can be seen on <https://www.deckshop.pro/deck/new-meta>) the penalization step will calculate the synergy score - **PBS** (Penalized Balance Score) as the following:

$$\text{rarity_diversity} \times (1 - |1 - \text{balance_anchor}|) \quad (2)$$

where the balance anchor is computed with the following notion:

- **One building.** In this case, we have 7 spare cards, which can be divided into 3 meta-groups: 3:4 , 2:5 and 1:6 spell-trace ratios. An approximate average would be **0.44**. we’ll set it as our balance anchor in order to penalize unbalanced decks.
- **None buildings.** The same as before, we have 8 spare cards and meta-ratios are 1:7, 2:6 and 3:5. In this case we’ll set our balance anchor as **0.355**.
- **Two buildings.** This approach is less common but exists in the game. We will denote our good ratios as 1:5 and 2:4, which means our balance anchor is **0.375**.
- **Otherwise.** The provided deck is not balanced at all (too much buildings) so we don’t penalize it - *“if it works, it works”*

Winner Elixir Score A score assigned for the winner’s average elixir usage based on a distance function:

$$f(x) = \begin{cases} 1 & \text{if } L \leq x \leq U \\ e^{-|x-L|} & \text{if } x < L \\ e^{-|x-U|} & \text{if } x > U \end{cases}$$

where Δ_0 is our hyper-parameter we tuned according to how much decks we penalize by being far from the range. In other words, we shrink the acceptable range of average deck elixir. After fine tuning, we chose:

$$\Delta_0 = 0.2$$

$L = \text{avg} - \Delta_0$ is the lower bound

$U = \text{avg} + \Delta_0$ is the upper bound.

Final Deck Strength Score We integrated multiple features into a single final deck strength score, which combines deck strength, card balance, and synergy (PBS) metrics:

$$\begin{aligned} \text{final_score} = & 0.25 \times \text{winner.win_rate} + \\ & 0.25 \times \text{winner.elixir_score} + \\ & 0.15 \times \frac{\text{max_card_level} + \text{min_card_level}}{2} + \\ & 0.15 \times \text{PBS} + \\ & 0.10 \times (1 - \text{level_variance}) + \\ & 0.10 \times \text{winner.avg_card_level} \end{aligned}$$

This score provides a comprehensive ranking of deck strength, balancing raw card levels with synergy and historical performance.

2.4 Analysis, Prediction and Inference

2.4.1 PCA

Definition: Principal component analysis (PCA) is a standard tool in modern data analysis. It is a method for extracting relevant information from confusing data sets. It does that by identifying the principal components, which are orthogonal directions of maximum variance in the dataset. The transformation is based on computing the eigenvalues and eigenvectors of the data covariance matrix, allowing for the projection of the data into a new feature space with reduced dimensionality[5].

Application: Our dataset consists of 78 features (before preparing it for PCA - cleaning of ID columns) so we want to reduce the high dimensional. However, we want to set up a threshold for how much of the data we want to retain, so we used a common heuristic of **preserving 95% of the variance** so our new target is to find the minimum set of components (linear combination of the original features) that capture at least 95% of the variance in the data. For that case, we'll denote cumulative explained variance as PEV , which is calculated in the following way:

$$PEV_k = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^N \lambda_i} \quad (3)$$

where λ_i are the eigenvalues of the covariance matrix, N is the total of found components and k is the number of principal components chosen. To determine the optimal number of components, we compute PCA on the standardized dataset, calculate the cumulative explained variance as a function of the

number of components and identify the smallest k such that $PEV_k \geq 0.95$ (i.e., the point where 95% of the variance is retained).

Inference:

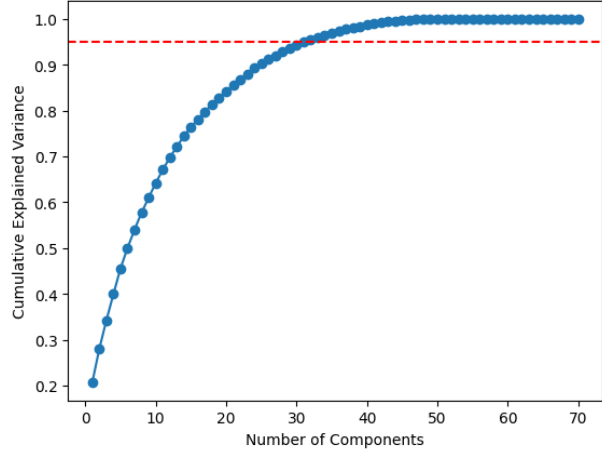


Figure 7: Cumulative Explained Variance vs. Number of Components

Running and tuning the PCA algorithm, we find that the minimum set of n components which pass the variance threshold is **32 components** which means, by using 32 components, we retain at least 95% of the variance. Derived from that, we run the PCA algorithm with $n_components = 32$ to reduce our dataset from 78 features to 32.

Following dimensionality reduction, we applied **K-Means clustering** ($k = 4$, chosen based on silhouette scoring) to the PCA-transformed dataset. The clusters revealed four distinct player archetypes:

- **Aggressive Swarmers:** Players who deploy high troop counts, rely on fast-cycle decks, and overwhelm opponents.
- **Experienced Strategists:** Players who optimize deck selection and elixir management to achieve consistent wins.
- **Legendary Reliants:** Players who construct their strategy as high-rarity legendary cards, utilizing their uniqueness for advantage.
- **Inefficient Builders:** Players with strong decks but suboptimal execution, often mismanaging elixir and deploying inefficient combinations.

To assess the effectiveness of PCA in clustering, we visualized the first two principal components, revealing well-separated clusters in 2D space:

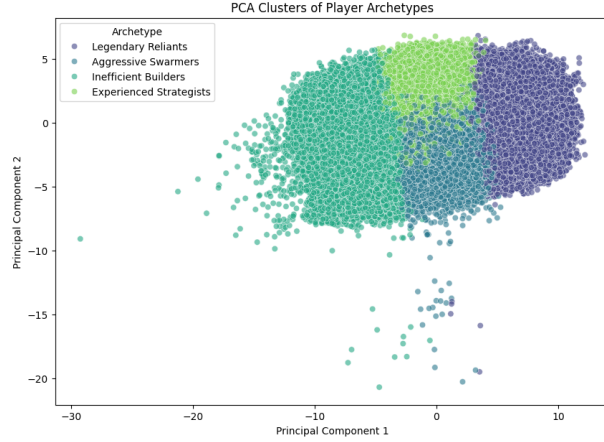


Figure 8: PCA projection. Each point represents a battle instance, colored by the identified player archetypes. The clustering reveals distinct playstyles, with Aggressive Swarmers (green) on the top, Legendary Reliants (teal) on the left, Experienced Strategists (blue) on the bottom and Inefficient Builders (purple) occupying the rightmost region.

Further analysis was conducted to determine which PCs contributed most to cluster separation by measuring their mean and assigning the "right" values to their archetype deck. We identified key gameplay attributes differentiating each archetype:

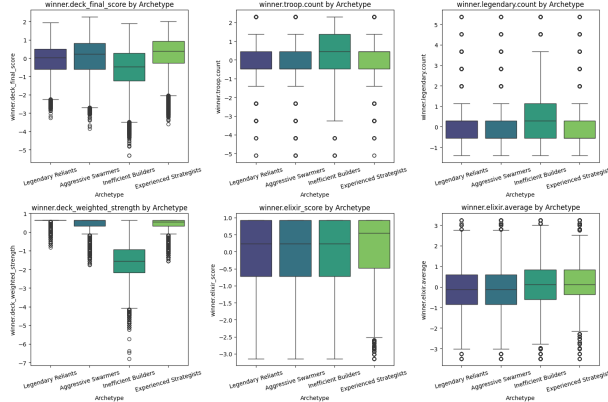


Figure 9: Feature distribution across player archetypes. The boxplots illustrate key deck properties, such as overall strength, troop count, legendary card usage, and elixir management.

Key Observations: Deck Final Score: Experienced Strategists and Inefficient Builders have the highest deck scores, while Aggressive Swarmers have the weakest, and Legendary Reliants show moderate strength. This suggests that strong deck compositions do not always translate to optimal performance,

as execution also plays a role.

Troop Count: Aggressive Swarmers deploy the highest number of troops, followed closely by Legendary Reliants. This highlights that Swarmers and Reliants favor high-troop strategies, whereas strategists rely on balanced, controlled decks.

Legendary Count: As expected, Legendary Reliants use the most legendary cards, while Aggressive Swarmers have the least due to their high elixir costs being incompatible with fast-cycle strategies. Strategists and Inefficient Builders show moderate legendary usage, indicating that legendary cards are not necessarily required for strong strategic play.

Deck Weighted Strength: Experienced Strategists and Inefficient Builders possess the strongest decks, whereas Aggressive Swarmers have the weakest. However, strong decks alone do not guarantee success—Inefficient Builders struggle with execution, despite having well-structured decks.

Elixir Score: Inefficient Builders display high variability in elixir score, indicating inconsistent resource management. Experienced Strategists maintain stable elixir control, while Aggressive Swarmers expend elixir more aggressively. This suggests that mismanagement of elixir may contribute to Inefficient Builders' suboptimal performance.

Elixir Average: Inefficient Builders show greater variation in elixir usage, whereas Aggressive Swarmers use elixir more consistently. Elixir management is a critical factor in competitive success, with Swarmers excelling in efficient troop cycling while Builders exhibit inconsistent spending.

Deck Composition Insights: To further validate the clustering, we extracted the most frequently used decks within each archetype. The results showed clear distinctions:

- **Aggressive Swarmers** favor fast-cycle decks (e.g., *Hog Rider*, *Musketeer*, *Ice Spirit*).
- **Legendary Reliants** heavily utilize high-rarity cards (e.g., *Mega Knight*, *Inferno Dragon*).
- **Experienced Strategists** prefer control decks such as *X-Bow* and *P.E.K.K.A*.
- **Inefficient Builders** tend to construct heavy decks (e.g., *Golem*, *Lava Hound*) but often misuse them.

Further Analysis: A natural next step is to leverage deep learning techniques to further refine deck optimization. One promising approach is the use of **Variational Autoencoders (VAEs)** to encode

decks into a latent representation, allowing us to generate optimized decks tailored to each archetype. This would enable the generation of new decks that adhere to the strategic tendencies of a given archetype while optimizing performance metrics such as elixir efficiency.

Alternatively, we can train a deep classification model to predict an archetype given a deck composition. This would allow for automatic deck-to-archetype labeling and facilitate the identification of weaknesses. Given a deck, the model could determine which archetype it belongs to and suggest modifications that make it more resistant to counters within the same cluster.

These methods represent an extension of our PCA-based clustering, allowing for automated deck recommendation, archetype classification, and strategic optimization.

Conclusion: The application of PCA successfully reduced the dataset from **78 features to 32**, preserving **95% of the variance** while significantly improving computational efficiency for further subsequent analysis. The clustering results revealed 4 distinct player archetypes—*Aggressive Swarmers*, *Experienced Strategists*, *Legendary Reliants*, and *Inefficient Builders*—each characterized by unique deck compositions and playstyles. Notably, the **Legendary Reliants** archetype exhibited the highest win rate (**up to 64.89%**), indicating that reliance on high-rarity cards can be a strong predictor of success when managed effectively. Conversely, **Inefficient Builders**, despite possessing strong decks, demonstrated **suboptimal win rates** due to poor elixir management and misplays. The **Experienced Strategists** archetype maintained **consistent win rates** ($\sim 57\%$), suggesting that balanced deck composition and controlled play yield competitive advantages. The PCA-transformed space also provided clear separation between archetypes, confirming that key gameplay factors—such as **troop deployment frequency**, **elixir efficiency**, and **deck synergy**—serve as primary differentiators among player types. Future work could explore the integration of **deep learning models**, such as autoencoders, to further refine archetype classification and optimize deck recommendations.

2.4.2 Deck Similarity

Deck compositions in battles can vary significantly, and identifying patterns in deck similarity is crucial for understanding dominant strategies. To visualize these relationships, we use the

MDS(Multi-Dimensional Scaling)[2] dimensionality reduction techniques.

Why Use MDS Instead of t-SNE? MDS is preferred over t-SNE for our dataset due to its ability to preserve **global pairwise distances** in a lower-dimensional space. Our dataset consists of numerical features such as `deck_weighted_strength` and `winner_deck_final_score`, which encode meaningful quantitative relationships between battle decks. Unlike t-SNE, which is optimized for local neighborhood preservation[3], MDS ensures that the **relative distances between battles remain consistent** in the reduced space, making it more suitable for understanding overall structural similarities between decks rather than isolated clusters. Given our goal of visualizing the relationships between different battle decks while maintaining their global structure, MDS provides a more stable and interpretable representation of the data:

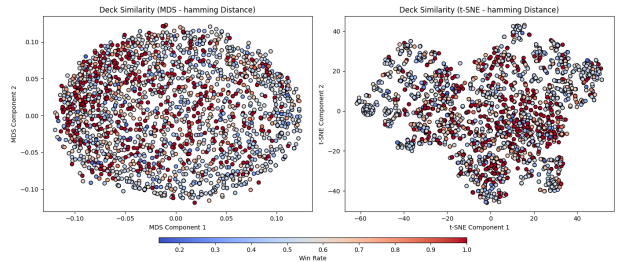


Figure 10: MDS vs. t-SNE: Deck comparison highlighted by the win-rate feature

The first difference key is **Better Point Distribution**, the t-SNE plot shows tightly packed clusters where decks are grouped into separate regions, while MDS displays points more evenly across the space. This clustering in t-SNE makes it difficult to distinguish between high and low win rates, when they overlap in dense areas, as seen in the circular cluster on the left side of the t-SNE plot and in the lower center and right regions for example. A second difference key is **Clearer Win Rate Patterns**, MDS shows more interpretable patterns of win rates with reasonable spacing between points (for instance in the left part of MDS plot). This spacing helps us see gradual changes in deck performance instead of abrupt transitions between clusters, making it easier to understand how deck compositions relate to win rates and less prone to overfitting when generalizing.

Sampling: To optimize the selection of battles for MDS, we ensure that our dataset retains enough points of interest while maintaining computational

feasibility. To achieve this, we created the feature `winner.high_win_rate`, a boolean indicator that identifies whether the winning deck has a win rate above 75%. This allows us to prioritize high-performance decks in our sample.

Given that the probability of a deck having a win rate above 75% is:

$$p(\text{deck_win_rate} > 0.75) = 0.247$$

we determine the required sample size, n , based on the expectation:

$$E[\text{deck_win_rate} > 0.75] = n \times p$$

Assuming we require at least $k = 501$ high-win-rate decks in our sample, we estimate the necessary sample size as:

$$n = \left\lceil \frac{k}{p} \right\rceil = \left\lceil \frac{501}{0.247} \right\rceil \approx 2025$$

Thus, we will randomly sample approximately 2025 battles while ensuring that the proportion of high-win-rate decks aligns with the overall dataset. If necessary, we will dynamically adjust n based on performance constraints.

Methodology: We define deck similarity using the **Hamming distance**, measuring the number of differing cards between two decks. The reason we use Hamming distance and not jaccard or cosine distances is due to the nature of Hamming distance, which is the number of positions at which the corresponding symbols are different between two vectors of equal length. Due to that, and to the computational benefits, we decided to use Hamming distance. We then proceed to create our pairwise-distances matrix but we can't include our whole records (2M rows) so we sample our data. We then apply **MDS** to project the high-dimensional deck similarity space into a two-dimensional representation.

Experiment 1 In this experiment, we try to find out if a relation exists between running MDS on deck similarity and highlighting different special features such as `winner.deck.final.score`, `winner.win_lose_ratio` and `winner.elixir.average`. As can be seen on Figure 12, The left plot of `deck.final.score` shows a concentration of high-scoring decks in the upper left region, where low-scoring decks are largely absent. The right plot exhibits a strong overlap, suggesting that high `deck.final.score` correlates with a higher win rate, making it a reliable predictor of deck effectiveness. The left plot of `win_lose_ratio`

and the right plot, show strong alignment. This indicates that high-performing players tend to use high-performing decks.

Experiment 2 In this experiment, we check the left cluster we found on our win rate MDS graph:

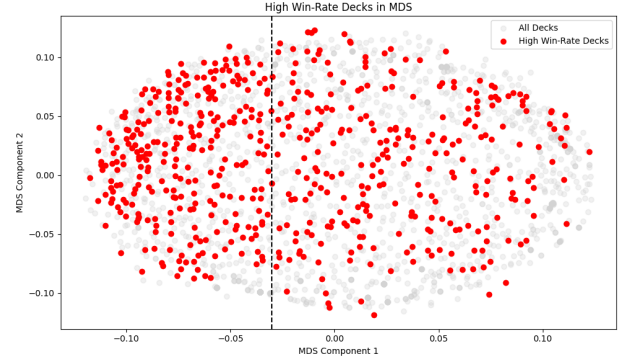


Figure 11: MDS on deck comparison highlighted by high ($> 75\%$) win rate

We choose our separator on the first MDS component at -0.03 and we will use table 5 to interpret our results. We observe that the total percentage of high-win-rate decks on the left side of the boundary (the rightmost column of our results) is approximately 52%, despite the dividing boundary splitting the plot unevenly, with the left side covering only about a third of the total area. This discrepancy strongly suggests the presence of a distinct cluster of decks in the MDS space with a high win rate in the left side, indicating that deck placement is not random but structured around inherent similarities in strategy and composition. Among all archetypes, *Inefficient Builders* exhibit the most pronounced clustering behavior on the left side. The majority of successful decks within this archetype are positioned on the left side of the boundary, suggesting that these decks share a high degree of structural similarity compared to other archetypes. A key observation is that most high-win-rate decks within the *Inefficient Builders* archetype are concentrated in the cluster, reinforcing the idea that decks following this strategy tend to succeed when they adhere to a specific composition pattern more than others (even of the same type).

Conclusions A deeper look suggests that the strength of *Inefficient Builders* decks does not stem solely from individual heavy cards but rather from the core structure of the deck itself as points placed in the MDS plot are placed according to the hamming distance between the decks and therefore, decks that

are close to each other tend to be similar at their core. This indicates that a winning *Inefficient Builders* deck typically requires only minor modifications (one or two card changes) to be effective. Within the left side of the boundary, this archetype surpasses the 57% win-rate threshold, a significantly higher success rate than *Inefficient Builders* decks found outside the cluster which confirms that we have successfully identified a structured meta within the MDS space—a group of *Inefficient Builders* decks that are not only closely related but also consistently outperform their counterparts outside this cluster.

Further Steps This finding opens the door for further granular analysis. By extracting highly dense subclusters within the *Inefficient Builders* group and analyzing the specific card choices that differentiate the most successful decks, we can refine our understanding of what makes these decks viable. A logical next step is to identify and optimize key deck compositions that mitigate the known weaknesses of *Inefficient Builders* while preserving its core strengths, potentially leading to the development of an optimized meta deck capable of countering existing inefficiencies.

3 Impact and Recommendations

- Our DAG analysis, using a 0.55 correlation threshold, revealed key insights: Win/Loss Ratio: Long-term success is heavily influenced by deck selection, with win rate serving as a key predictor. We successfully modeled the formula to achieve a strong correlation with the win/loss ratio. and found the main cause of Immediate success correlates with maximizing damage to the Princess and King Towers. These findings help players refine deck strategies and assist game developers in balancing mechanics for fairer, more engaging gameplay.
- The findings from our PCA and clustering analysis offer valuable insights for both competitive players and game balance designers. By identifying distinct archetypes and their associated win rates, this study provides a foundation for **personalized deck optimization strategies**, allowing players to refine their playstyles based on data-driven recommendations. For instance, *Inefficient Builders* could benefit from **improved elixir management heuristics**, while *Aggressive Swarmers* may enhance their performance

by optimizing **early-game pressure tactics**. Additionally, game developers can utilize these insights to ensure balanced gameplay mechanics, preventing over-reliance on high-rarity cards while maintaining strategic diversity. Future work could explore the integration of RL agents to simulate optimal decision-making pathways, further refining strategic recommendations for different playstyles.

- Optimizing deck final score is helpful in obtaining higher win rates, as high-scoring decks cluster in successful MDS regions. Players should focus on well-synergized, high-value cards and refine compositions based on performance data. The strong correlation between win/loss ratio and deck win rate highlights the importance of structured, high-performing decks with clear win conditions. Elixir cost optimization also matters, as successful decks cluster around specific elixir distributions. Maintaining an efficient balance prevents inefficiencies—avoiding decks that are too slow or too fast but lacking defense. Finally, certain archetypes such as *Inefficient Builders* decks succeed when built around a strong core. Their clustering in MDS space suggests that optimizing base deck composition with targeted adjustments leads to consistently high performance. Understanding these factors helps players refine strategies and adapt to evolving metas.

4 Conclusion

This study explored the impact of player behavior features on game performance using data-driven methodologies, including Exploratory Data Analysis (EDA), Principal Component Analysis (PCA), and clustering techniques such as K-Means. Our findings highlighted key behavioral archetypes—*Aggressive Swarmers*, *Experienced Strategists*, *Legendary Reliants*, and *Inefficient Builders*—each characterized by distinct deck compositions and playstyles.

Through PCA, we successfully reduced the dataset’s dimensionality while retaining 95% of the variance, enabling effective clustering of players based on their strategic tendencies. The clustering results provided actionable insights, such as the high effectiveness of well-balanced, high-rarity decks in competitive gameplay and the challenges faced by players with suboptimal elixir management.

Additionally, our MDS-based deck similarity analysis revealed structural patterns in deck compositions, confirming that decks with high win rates

sometime share specific card combinations and strategic synergies. This insight is crucial for both players seeking to optimize their decks and game developers aiming to balance mechanics effectively.

Future work could leverage advanced machine learning techniques, such as reinforcement learning and neural networks, to refine deck recommendations and strategic decision-making. Moreover, integrating real-time game analytics could further enhance the predictive power of our models, offering dynamic strategy adjustments tailored to player behavior.

By identifying key determinants of success in *Clash Royale*, this study contributes valuable knowledge to both competitive gaming communities and the broader field of game analytics. Our approach demonstrates the power of data-driven insights in shaping strategic decision-making.

5 References

- [1] Bwandowando. *Clash Royale Season 18 Dataset*. 2021. URL: <https://www.kaggle.com/datasets/bwandowando/clash-royale-season-18-dec-0320-dataset>.
- [2] Joseph B. Kruskal. “Multidimensional Scaling by Optimizing Goodness of Fit to a Nonmetric Hypothesis”. In: *Psychometrika* (1964). URL: <https://doi.org/10.1007/BF02289565>.
- [3] Geoffrey Hinton Laurens van der Maaten. “Visualizing Data using t-SNE”. In: (). URL: <https://www.jmlr.org/papers/volume9/vandermaaten08a/vandermaaten08a.pdf>.
- [4] Abhinav Shaw. *Clash Royale Dataset*. 2020. URL: <https://www.kaggle.com/datasets/abhinavshaw09/clash-royal-dataset>.
- [5] Jonathon Shlens. “The structure of performance and training in esports”. In: (). URL: <https://arxiv.org/pdf/1404.1100>.
- [6] “The Analytics Behind Esports: Data-Driven Strategies in Competitive Gaming”. In: (). URL: <https://www.statology.org/the-analytics-behind-esports-data-driven-strategies-in-competitive-gaming/>.
- [7] “The structure of performance and training in esports”. In: (). URL: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0237584>.

A Appendix.

A.1 MDS to find out deck similarity effect on selected features

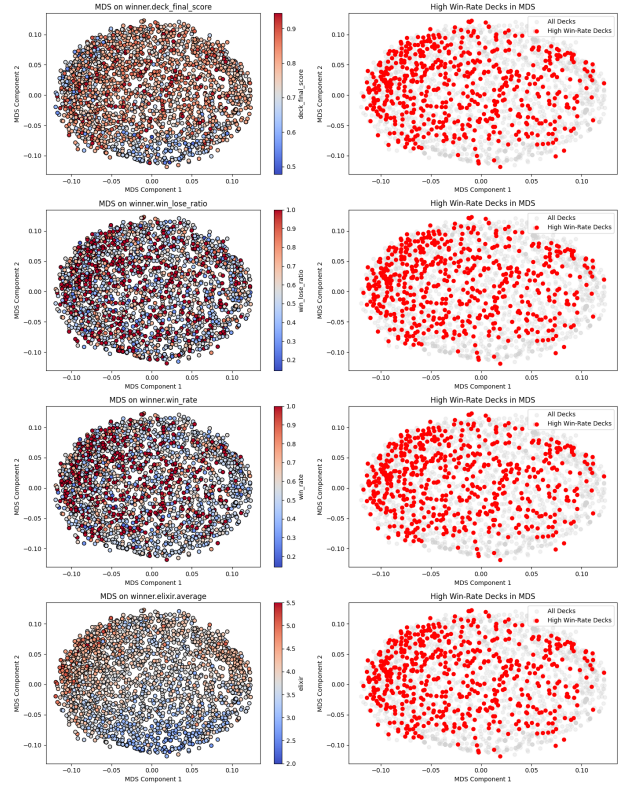


Figure 12: MDS on selected features

A.2 Deck Archetype Analysis

Archetype	Count	Win Rate	Deck Composition
Aggressive Swarmers	51	0.6037	Princess, Dark Prince, Ice Spirit, Goblin Gang, Inferno Tower, Rocket, Goblin Barrel, The Log
Aggressive Swarmers	51	0.5962	Knight, Skeleton Army, Dart Goblin, Skeleton Barrel, Mother Witch, Bomb Tower, Goblin Barrel, The Log
Aggressive Swarmers	95	0.5949	P.E.K.K.A, Dark Prince, Battle Ram, Electro Wizard, Bandit, Firecracker, Zap, Poison
Aggressive Swarmers	52	0.5938	Knight, Goblins, Hog Rider, Ice Spirit, Executioner, Fireball, The Log, Tornado
Aggressive Swarmers	54	0.5920	Knight, Archers, Ice Spirit, Bats, Mortar, Arrows, Rocket, The Log

Table 1: Top 5 Decks in Aggressive Swarmers Archetype

Archetype	Count	Win Rate	Deck Composition
Experienced Strategists	94	0.5949	P.E.K.K.A, Dark Prince, Battle Ram, Electro Wizard, Bandit, Firecracker, Zap, Poison
Experienced Strategists	139	0.5866	P.E.K.K.A, Baby Dragon, Dark Prince, Battle Ram, Electro Wizard, Bandit, Zap, Poison
Experienced Strategists	84	0.5761	Princess, Dark Prince, Goblin Gang, Bandit, Rascals, Rocket, Goblin Barrel, The Log
Experienced Strategists	83	0.5740	Giant, Prince, Dark Prince, Miner, Mega Minion, Electro Wizard, Zap, Poison
Experienced Strategists	533	0.5712	Skeleton Army, Prince, Royal Giant, Princess, Dart Goblin, Bats, Mega Knight, Goblin Barrel

Table 2: Top 5 Decks in Experienced Strategists Archetype

Archetype	Count	Win Rate	Deck Composition
Inefficient Builders	57	0.6325	Prince, Princess, Dark Prince, Goblin Gang, Bandit, Bats, Zap, The Log
Inefficient Builders	50	0.5866	P.E.K.K.A, Baby Dragon, Dark Prince, Battle Ram, Electro Wizard, Bandit, Zap, Poison
Inefficient Builders	63	0.5739	Balloon, Baby Dragon, Bowler, Lumberjack, Inferno Dragon, Freeze, The Log, Tornado
Inefficient Builders	401	0.5712	Skeleton Army, Prince, Royal Giant, Princess, Dart Goblin, Bats, Mega Knight, Goblin Barrel
Inefficient Builders	63	0.5669	Spear Goblins, Miner, Inferno Dragon, Bandit, Bats, Mega Knight, Skeleton Barrel, Zap

Table 3: Top 5 Decks in Inefficient Builders Archetype

Archetype	Count	Win Rate	Deck Composition
Legendary Reliants	61	0.6489	Giant, Skeleton Army, Musketeer, Sparky, Zappies, Arrows, Zap, Graveyard
Legendary Reliants	57	0.6404	Balloon, Barbarians, Lava Hound, Mega Minion, Skeleton Dragons, Tombstone, Arrows, Freeze
Legendary Reliants	59	0.6344	Archers, Giant, Minions, Barbarians, Bomber, Arrows, Graveyard, Giant Snowball
Legendary Reliants	84	0.6325	Prince, Princess, Dark Prince, Goblin Gang, Bandit, Bats, Zap, The Log
Legendary Reliants	52	0.6281	Giant, Mini P.E.K.K.A, Sparky, Mega Minion, Electro Wizard, Fireball, Zap, Heal Spirit

Table 4: Top 5 Decks in Legendary Reliants Archetype

Archetype	Left section count of high-win-rate points	Right section count of high-win-rate points	Percentage of left section high-win-rate points vs. archetype high-win-rate points	Percentage of left section high-win-rate points vs. total high-win-rate points	Percentage of left section high-win- rate points vs. archetype points
Aggressive Swarmers	70	91	43.48	25.45	11.93
Experienced Strategists	89	91	49.44	32.36	16.06
Inefficient Builders	112	84	57.14	40.73	23.05
Legendary Reliants	4	6	40.00	1.45	1.01

Table 5: Archetype Distribution in Left and Right Sections